

Lab Taks-6

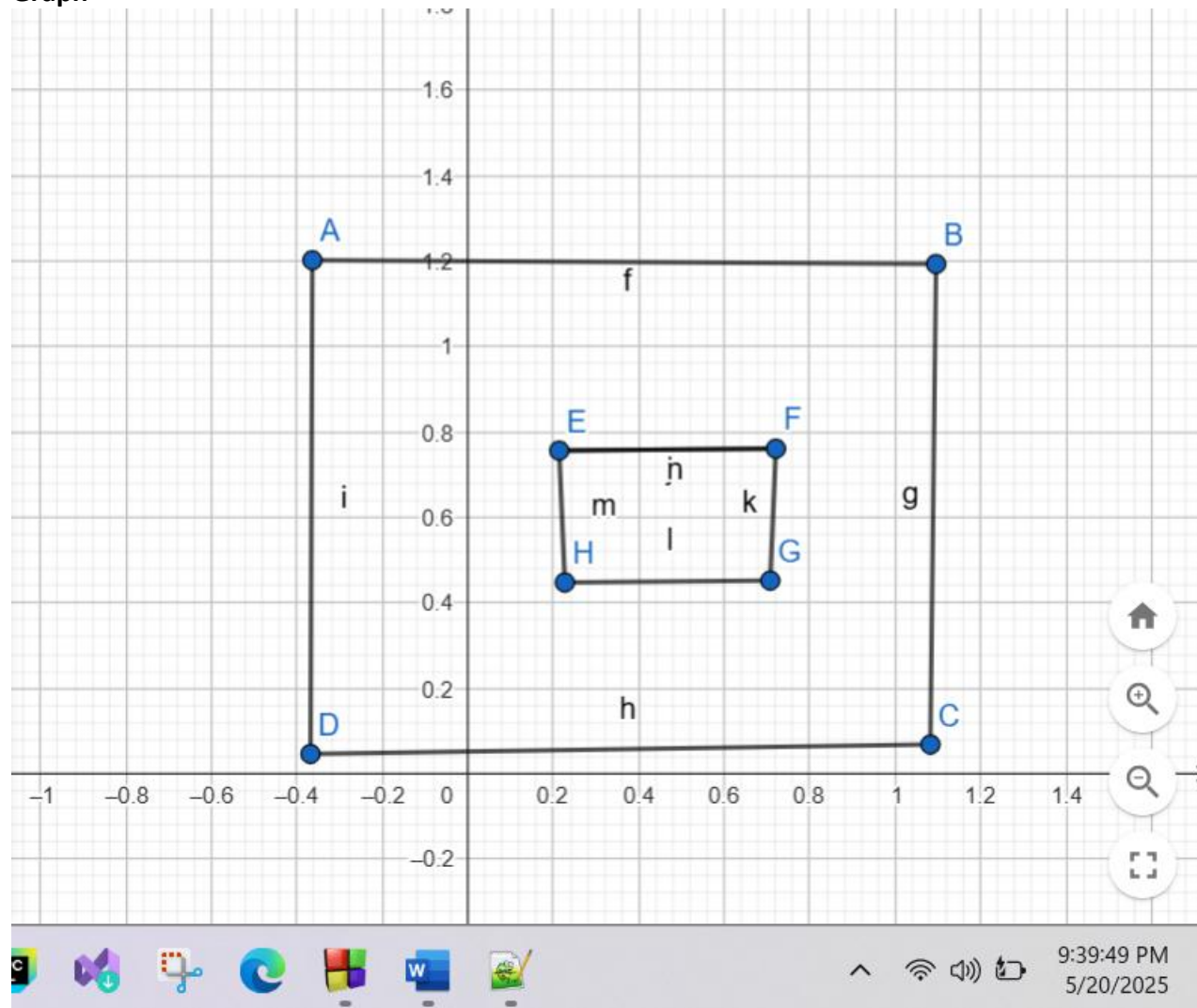
Submission Guidelines-

- Rename the file with your serial number only
- Must submit within the time discussed in class
- Must include resources for all the section in the table

Question-

Develop an animation that will change the background color of the window after 20ms. Use at least two different colors.

Graph-



Code-

```

#include <windows.h>
#include <GL/glut.h>

// Define color states (R, G, B)
float bgColors[][3] = {
    {0.2f, 0.3f, 0.8f}, // Blueish
    {0.9f, 0.6f, 0.1f}, // Orange
    {0.2f, 0.6f, 0.2f}, // Greenish
    {0.6f, 0.2f, 0.5f}  // Purple
};

int colorIndex = 0;
int totalColors = sizeof(bgColors) / sizeof(bgColors[0]);

void display() {
    // Clear screen with current background color
    glClear(GL_COLOR_BUFFER_BIT);

    // Optional: draw something to make it obvious
    glColor3f(1.0, 1.0, 1.0);
    glBegin(GL_QUADS);
        glVertex2f(-0.2f, -0.2f);
        glVertex2f(0.2f, -0.2f);
        glVertex2f(0.2f, 0.2f);
        glVertex2f(-0.2f, 0.2f);
    glEnd();

    glFlush();
}

void changeBackgroundColor(int value) {
    // Update background color
    float* color = bgColors[colorIndex];
    glClearColor(color[0], color[1], color[2], 1.0f);

    // Move to next color
    colorIndex = (colorIndex + 1) % totalColors;

    glutPostRedisplay();
    glutTimerFunc(20, changeBackgroundColor, 0);
}

void init() {
    glClearColor(bgColors[0][0], bgColors[0][1], bgColors[0][2], 1.0f); // Initial color
}

```

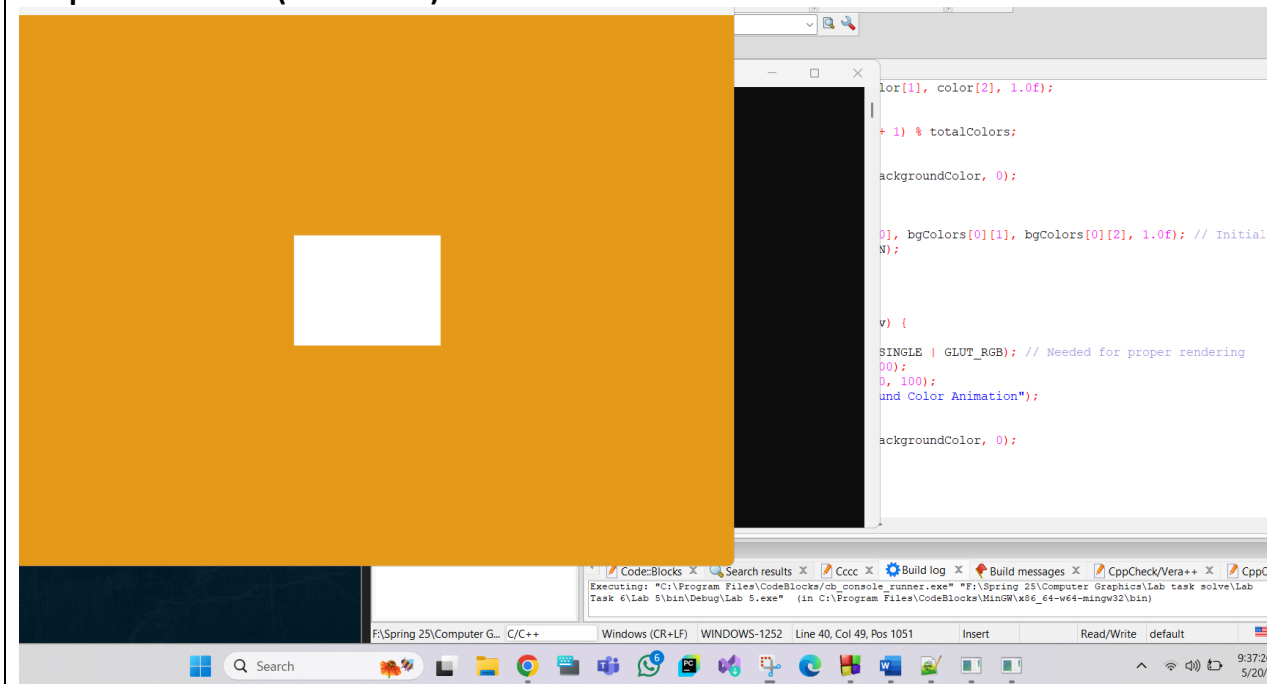
```

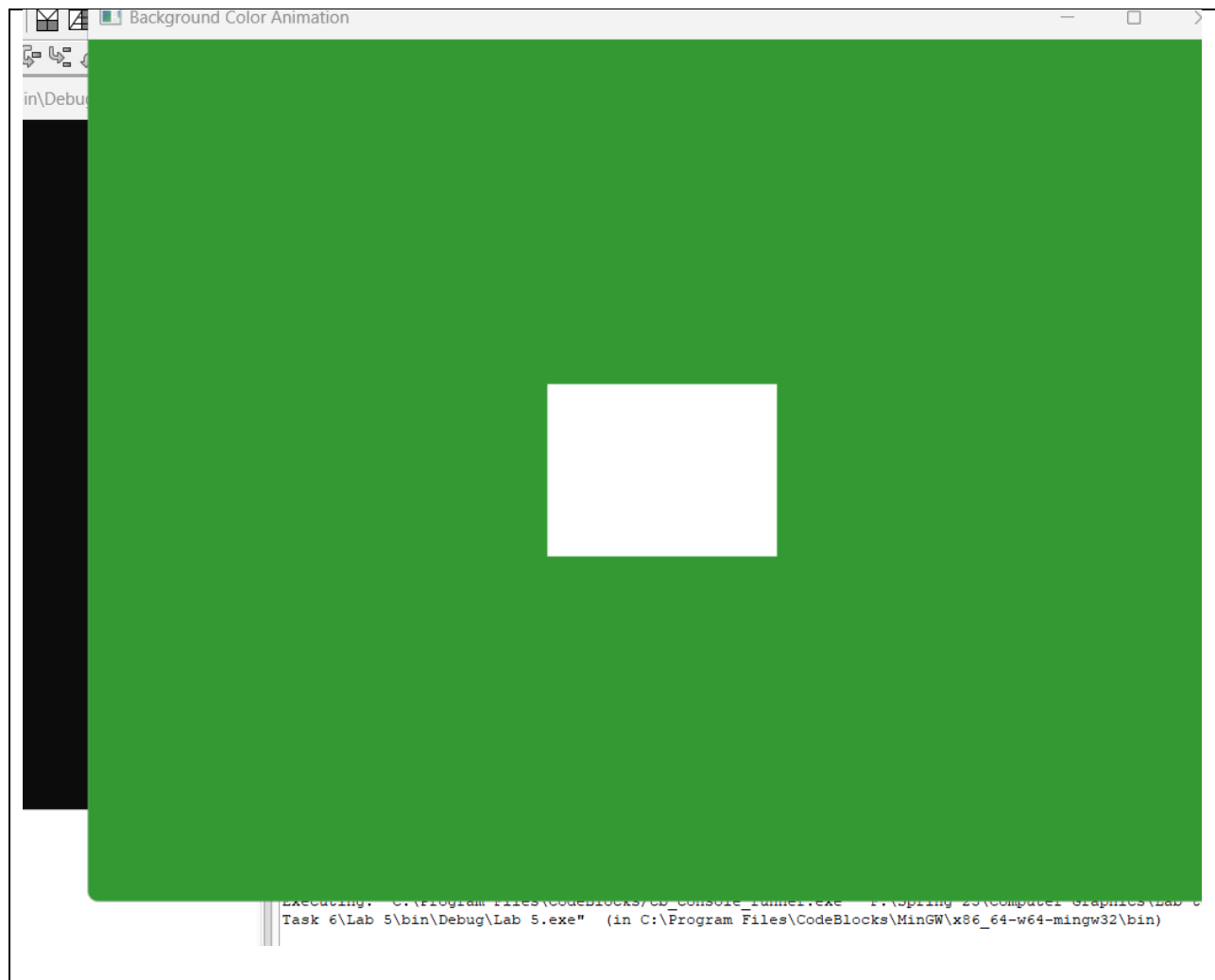
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
gluOrtho2D(-1, 1, -1, 1);
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB); // Needed for proper rendering
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("Background Color Animation");
    init();
    glutDisplayFunc(display);
    glutTimerFunc(20, changeBackgroundColor, 0);
    glutMainLoop();
    return 0;
}

```

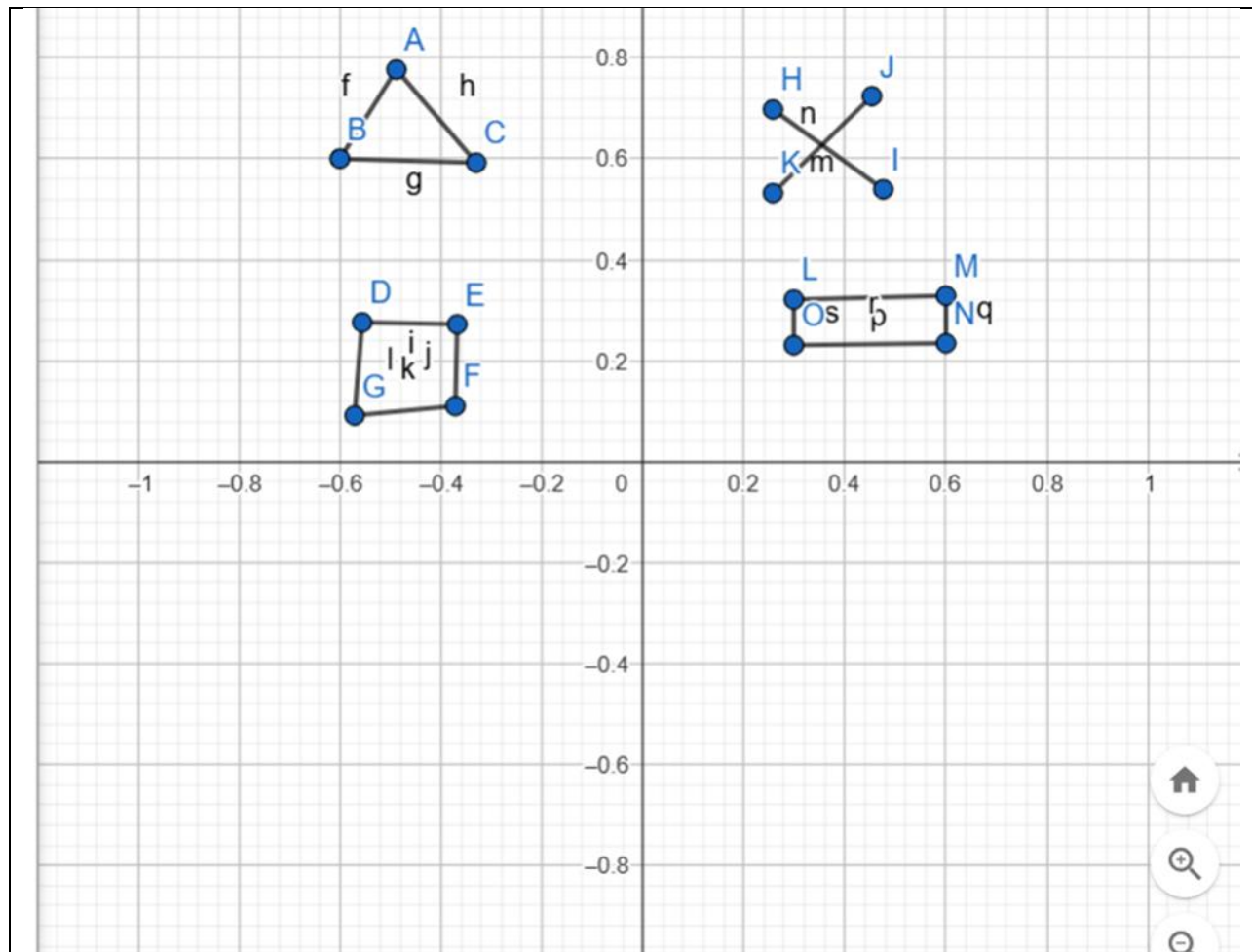
Output Screenshot (Full Screen)-



**Question-**

Develop an animation that will call four objects separately, each after 20 ms.

Graph-



Code-

```
#include <windows.h>
```

```
#include <GL/glut.h>
```

```
struct Object {
    float x, y;
    float dx, dy;
    int type; // 0: rectangle, 1: triangle, 2: star, 3: diamond
};
```

```
Object objects[4] = {
    {-0.8f, 0.6f, 0.01f, 0.0f, 0}, // Rectangle → right
    {0.8f, 0.4f, -0.01f, 0.0f, 1}, // Triangle ← left
    {-0.4f, -0.8f, 0.0f, 0.01f, 2}, // Star ↑ up
    {0.4f, 0.8f, 0.0f, -0.01f, 3} // Diamond ↓ down
};
```

```
void drawRectangle(float x, float y) {
    glBegin(GL_QUADS);
```

```

        glVertex2f(x - 0.05f, y - 0.03f);
        glVertex2f(x + 0.05f, y - 0.03f);
        glVertex2f(x + 0.05f, y + 0.03f);
        glVertex2f(x - 0.05f, y + 0.03f);
    glEnd();
}

void drawTriangle(float x, float y) {
    glBegin(GL_TRIANGLES);
        glVertex2f(x, y + 0.05f);
        glVertex2f(x - 0.05f, y - 0.05f);
        glVertex2f(x + 0.05f, y - 0.05f);
    glEnd();
}

void drawStar(float x, float y) {
    glBegin(GL_LINES);
        glVertex2f(x - 0.05f, y - 0.05f);
        glVertex2f(x + 0.05f, y + 0.05f);

        glVertex2f(x + 0.05f, y - 0.05f);
        glVertex2f(x - 0.05f, y + 0.05f);
    glEnd();
}

void drawDiamond(float x, float y) {
    glBegin(GL_POLYGON);
        glVertex2f(x, y + 0.05f);
        glVertex2f(x + 0.04f, y);
        glVertex2f(x, y - 0.05f);
        glVertex2f(x - 0.04f, y);
    glEnd();
}

void drawObject(Object& obj) {
    switch (obj.type) {
        case 0:
            glColor3f(0.2f, 0.6f, 1.0f); // Blue
            drawRectangle(obj.x, obj.y);
            break;
        case 1:
            glColor3f(1.0f, 0.4f, 0.2f); // Orange
            drawTriangle(obj.x, obj.y);
            break;
    }
}

```

```

        case 2:
            glColor3f(0.8f, 1.0f, 0.1f); // Yellow-green
            drawStar(obj.x, obj.y);
            break;
        case 3:
            glColor3f(0.7f, 0.2f, 1.0f); // Purple
            drawDiamond(obj.x, obj.y);
            break;
    }
}

void update(int value) {
    for (int i = 0; i < 4; ++i) {
        objects[i].x += objects[i].dx;
        objects[i].y += objects[i].dy;

        // Wrap-around
        if (objects[i].x > 1.1f) objects[i].x = -1.1f;
        if (objects[i].x < -1.1f) objects[i].x = 1.1f;
        if (objects[i].y > 1.1f) objects[i].y = -1.1f;
        if (objects[i].y < -1.1f) objects[i].y = 1.1f;
    }

    glutPostRedisplay();
    glutTimerFunc(16, update, 0); // ~60 FPS
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    for (int i = 0; i < 4; ++i) {
        drawObject(objects[i]);
    }

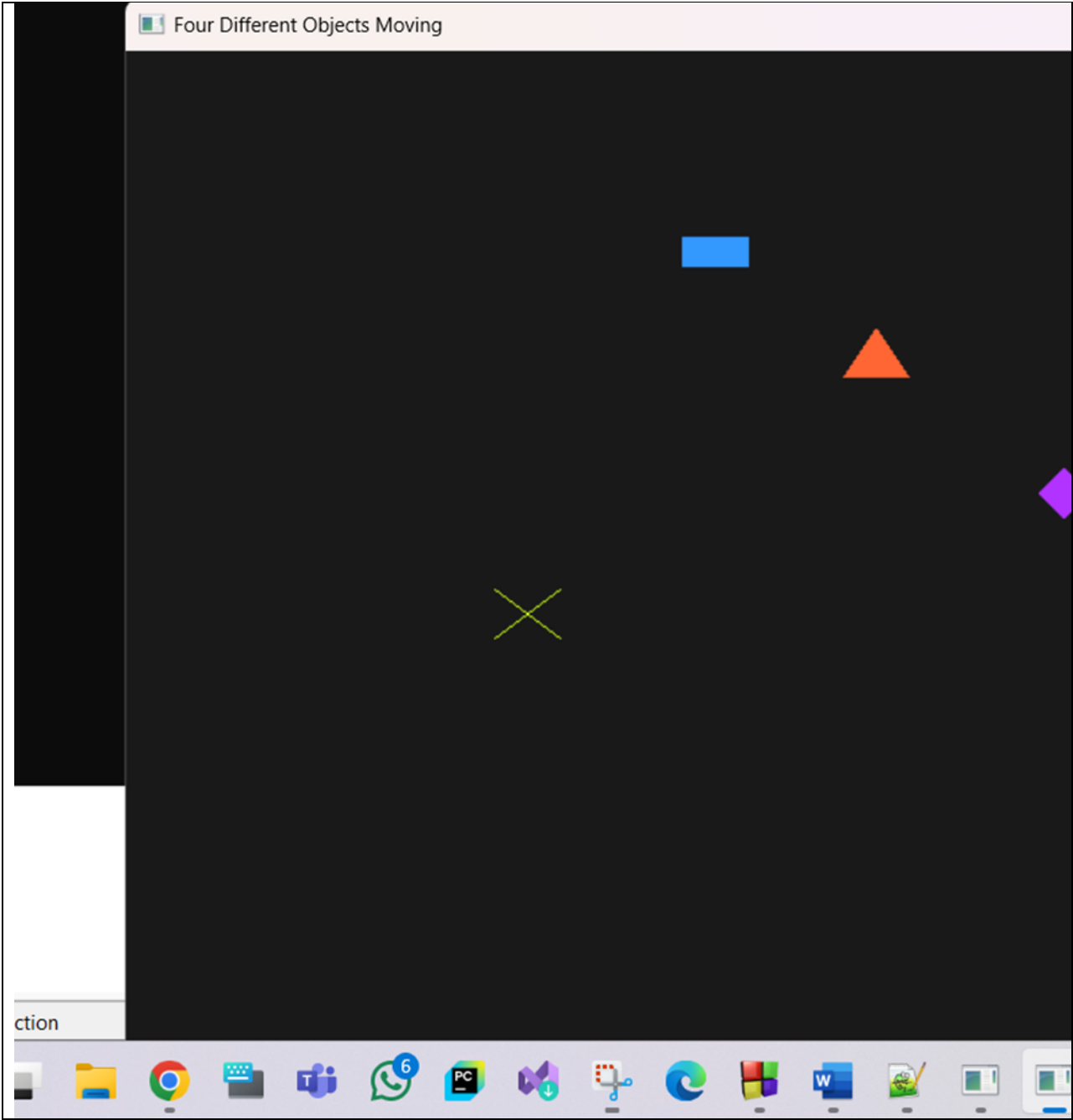
    glFlush();
}

void init() {
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f); // Dark background
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-1, 1, -1, 1);
}

```

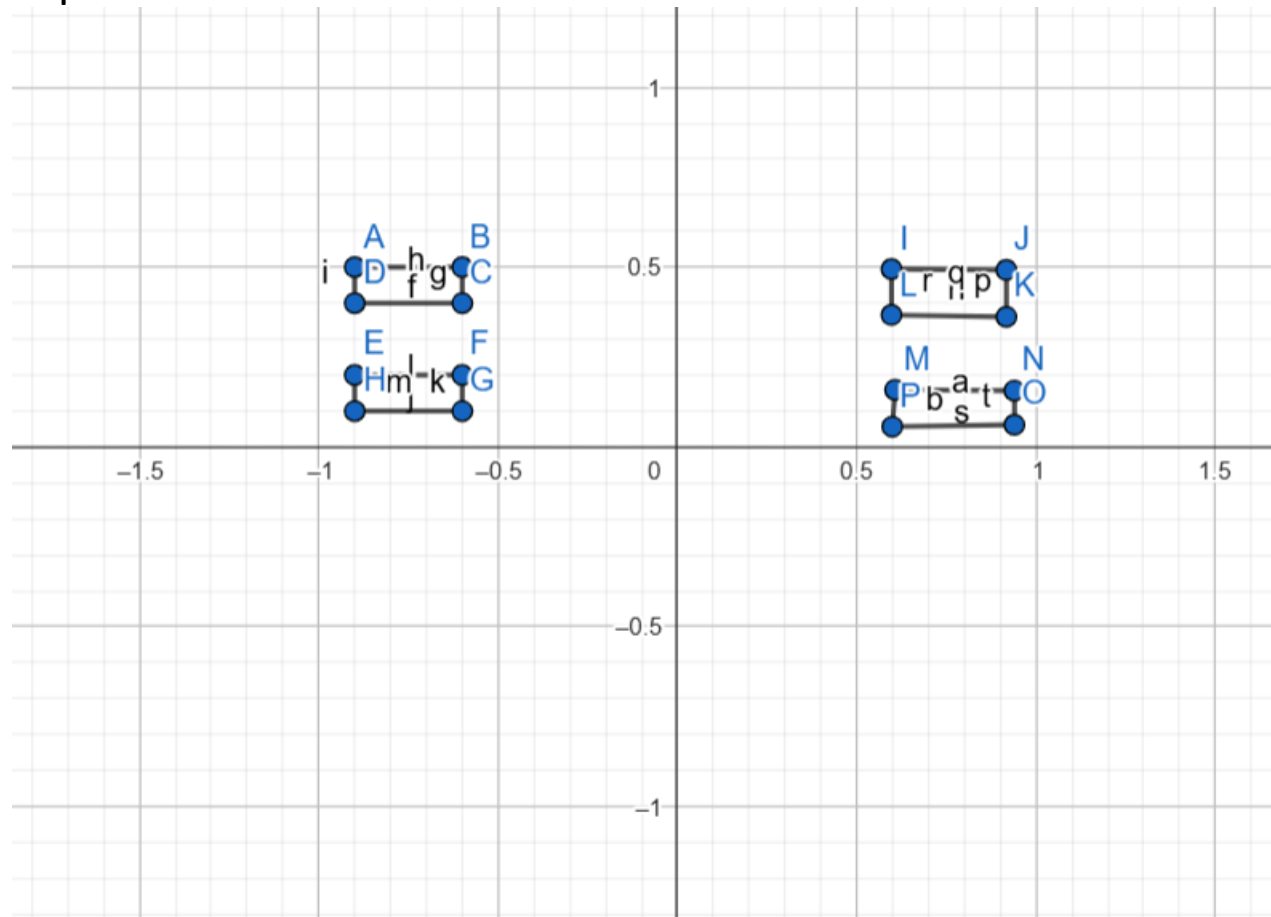
```
int main(int argc, char** argv) {  
    glutInit(&argc, argv);  
    glutInitWindowSize(800, 600);  
    glutCreateWindow("Four Different Objects Moving");  
    init();  
    glutDisplayFunc(display);  
    glutTimerFunc(0, update, 0);  
    glutMainLoop();  
    return 0;  
}
```

Output Screenshot (Full Screen)-



Question-

Develop a code that will have four different objects (keep it simple). The objects will move to the left, right, up and down in a loop.

Graph-**Code-**

```
#include <windows.h>
#include <GL/glut.h>

// Visibility flags for each object
bool show[4] = {false, false, false, false};

// Draw a rectangle at a position with color
void drawRectangle(float x, float y, float r, float g, float b) {
    glColor3f(r, g, b);
```

```

glBegin(GL_QUADS);
    glVertex2f(x - 0.1f, y - 0.05f);
    glVertex2f(x + 0.1f, y - 0.05f);
    glVertex2f(x + 0.1f, y + 0.05f);
    glVertex2f(x - 0.1f, y + 0.05f);
glEnd();
}

// Display function to render all visible objects
void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    if (show[0]) drawRectangle(-0.7f, 0.6f, 1.0f, 0.0f, 0.0f); // Red
    if (show[1]) drawRectangle( 0.7f, 0.6f, 0.0f, 1.0f, 0.0f); // Green
    if (show[2]) drawRectangle(-0.7f, -0.6f, 0.0f, 0.0f, 1.0f); // Blue
    if (show[3]) drawRectangle( 0.7f, -0.6f, 1.0f, 1.0f, 0.0f); // Yellow

    glFlush();
}

// Timer callback to enable object i and set next one
void revealObject(int i) {
    if (i < 4) {
        show[i] = true;
        glutPostRedisplay();
        glutTimerFunc(20, revealObject, i + 1);
    }
}

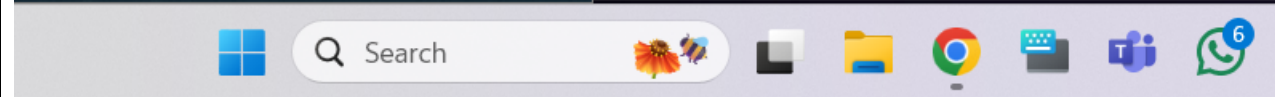
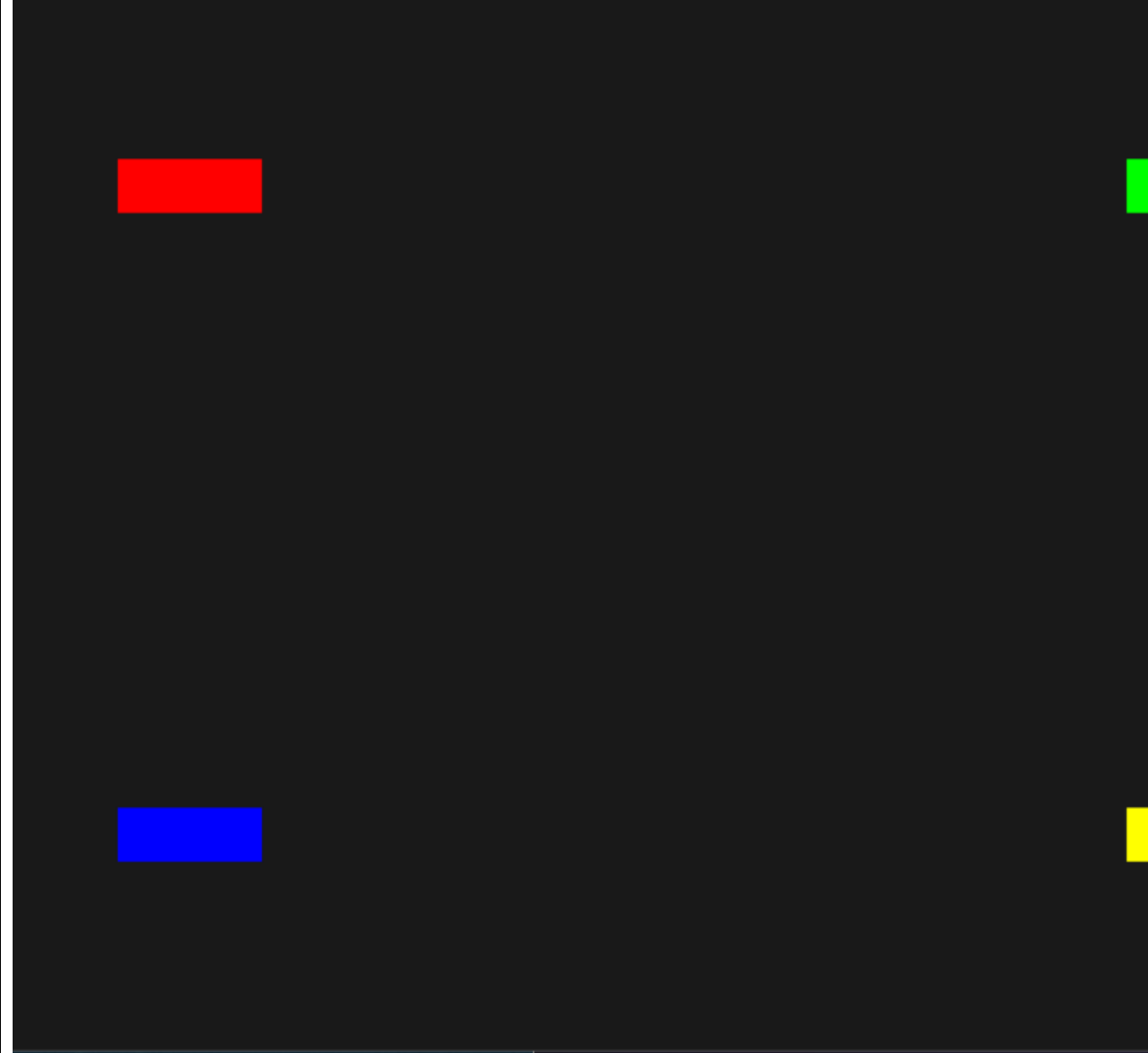
void init() {
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f); // Dark background
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-1.0, 1.0, -1.0, 1.0);
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Sequential Rectangle Animation");
    init();
    glutDisplayFunc(display);
    glutTimerFunc(20, revealObject, 0); // Start revealing rectangles
    glutMainLoop();
}

```

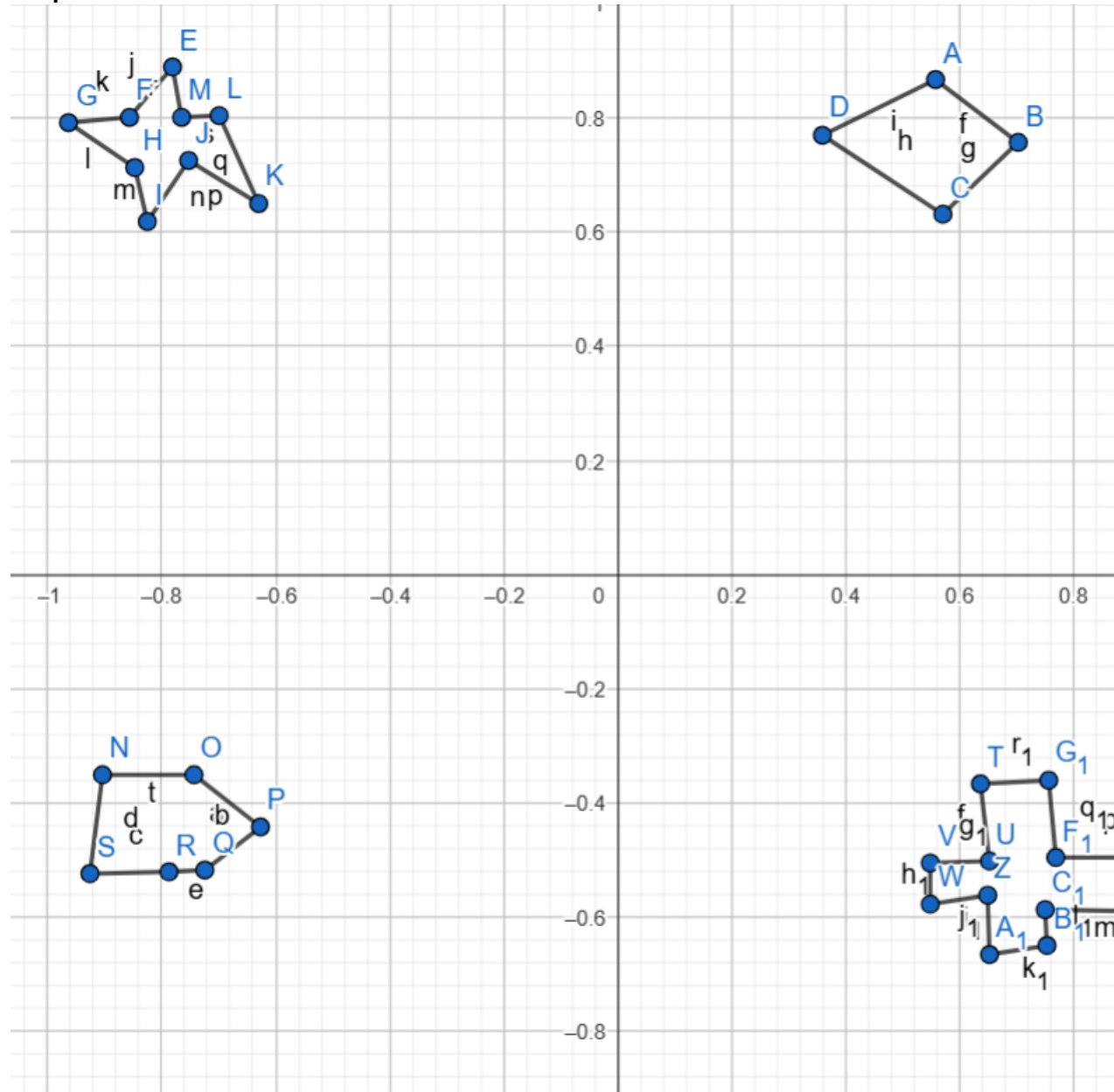
```
return 0;  
}
```

Output Screenshot (Full Screen)-



Question-

Develop a code that will have four different objects (keep it simple). Four different keys will be dedicated each objects. The objects will move to the left, right, up and down in a loop as the keys are pressed individually.

Graph-**Code-**

```
#include <windows.h>
#include <GL/glut.h>
#include <cmath>
```

```

float starX = -0.6f, starY = 0.6f;
float diamondX = 0.6f, diamondY = 0.6f;
float pentagonX = -0.6f, pentagonY = -0.6f;
float crossX = 0.6f, crossY = -0.6f;

float moveSpeed = 0.025f;

void drawStar(float x, float y) {
    glColor3f(0.6f, 0.1f, 0.7f); // Purple
    float radius = 0.05f;
    glBegin(GL_TRIANGLE_FAN);
    glVertex2f(x, y);
    for (int i = 0; i <= 10; i++) {
        float angle = i * 3.14159f / 5.0f;
        float r = (i % 2 == 0) ? radius : radius / 2;
        glVertex2f(x + cos(angle) * r, y + sin(angle) * r);
    }
    glEnd();
}

void drawDiamond(float x, float y) {
    glColor3f(1.0f, 0.5f, 0.0f); // Orange
    glBegin(GL_QUADS);
        glVertex2f(x, y + 0.06f);
        glVertex2f(x + 0.06f, y);
        glVertex2f(x, y - 0.06f);
        glVertex2f(x - 0.06f, y);
    glEnd();
}

void drawPentagon(float x, float y) {
    glColor3f(0.2f, 0.8f, 0.2f); // Green
    glBegin(GL_POLYGON);
    for (int i = 0; i < 5; i++) {
        float angle = i * 2.0f * 3.14159f / 5.0f;
        glVertex2f(x + cos(angle) * 0.05f, y + sin(angle) * 0.05f);
    }
    glEnd();
}

void drawCross(float x, float y) {
    glColor3f(0.9f, 0.1f, 0.1f); // Red
    float size = 0.02f;

```

```

glBegin(GL_QUADS);
    glVertex2f(x - size, y + 3 * size);
    glVertex2f(x + size, y + 3 * size);
    glVertex2f(x + size, y - 3 * size);
    glVertex2f(x - size, y - 3 * size);

    glVertex2f(x - 3 * size, y + size);
    glVertex2f(x + 3 * size, y + size);
    glVertex2f(x + 3 * size, y - size);
    glVertex2f(x - 3 * size, y - size);
glEnd();
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    drawStar(starX, starY);
    drawDiamond(diamondX, diamondY);
    drawPentagon(pentagonX, pentagonY);
    drawCross(crossX, crossY);
    glFlush();
}

void keyboard(unsigned char key, int x, int y) {
    switch (key) {
        // Star (WASD)
        case 'w': starY += moveSpeed; break;
        case 's': starY -= moveSpeed; break;
        case 'a': starX -= moveSpeed; break;
        case 'd': starX += moveSpeed; break;

        // Diamond (IJKL)
        case 'i': diamondY += moveSpeed; break;
        case 'k': diamondY -= moveSpeed; break;
        case 'j': diamondX -= moveSpeed; break;
        case 'l': diamondX += moveSpeed; break;

        // Pentagon (TFGH)
        case 't': pentagonY += moveSpeed; break;
        case 'g': pentagonY -= moveSpeed; break;
        case 'f': pentagonX -= moveSpeed; break;
        case 'h': pentagonX += moveSpeed; break;

        // Cross (UYMO)
        case 'u': crossY += moveSpeed; break;
    }
}

```

```

        case 'm': crossY -= moveSpeed; break;
        case 'y': crossX -= moveSpeed; break;
        case 'o': crossX += moveSpeed; break;
    }

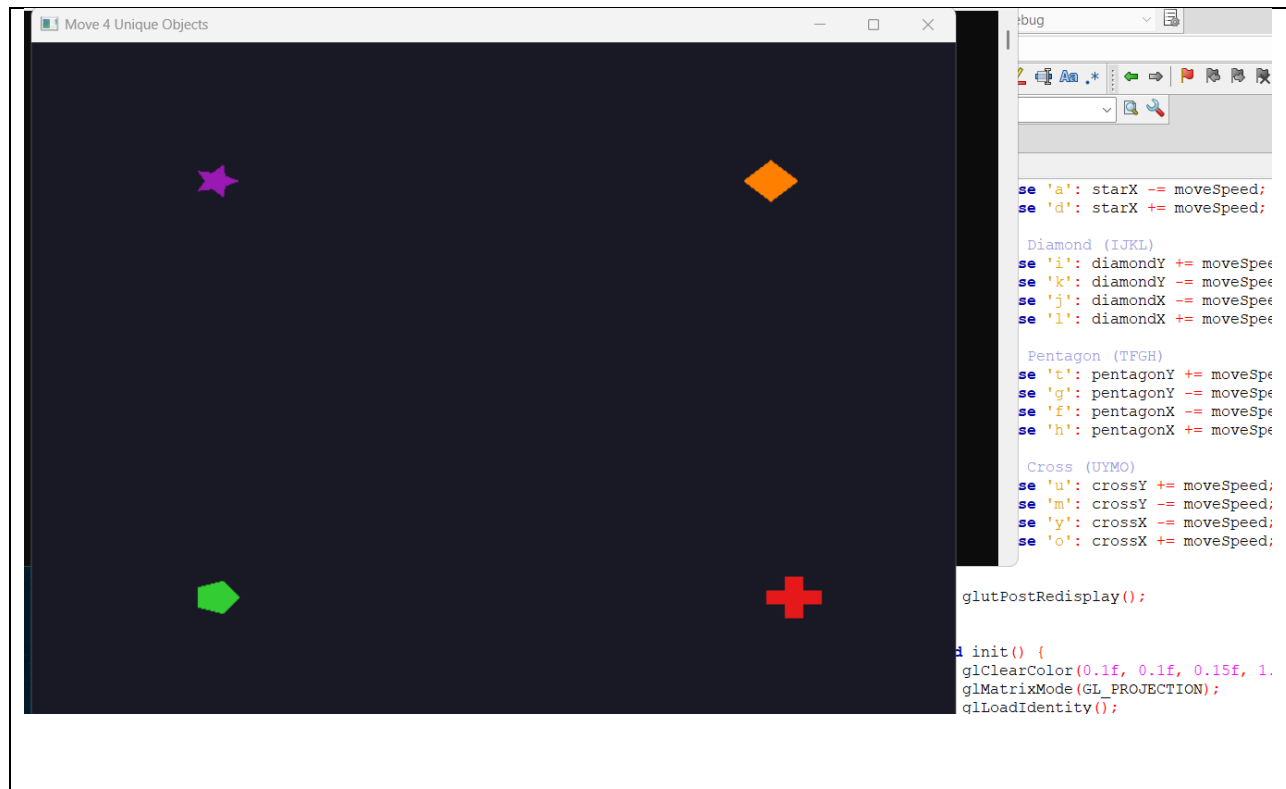
    glutPostRedisplay();
}

void init() {
    glClearColor(0.1f, 0.1f, 0.15f, 1.0f); // Dark background
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-1, 1, -1, 1);
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Move 4 Unique Objects");
    init();
    glutDisplayFunc(display);
    glutKeyboardFunc(keyboard);
    glutMainLoop();
    return 0;
}

```

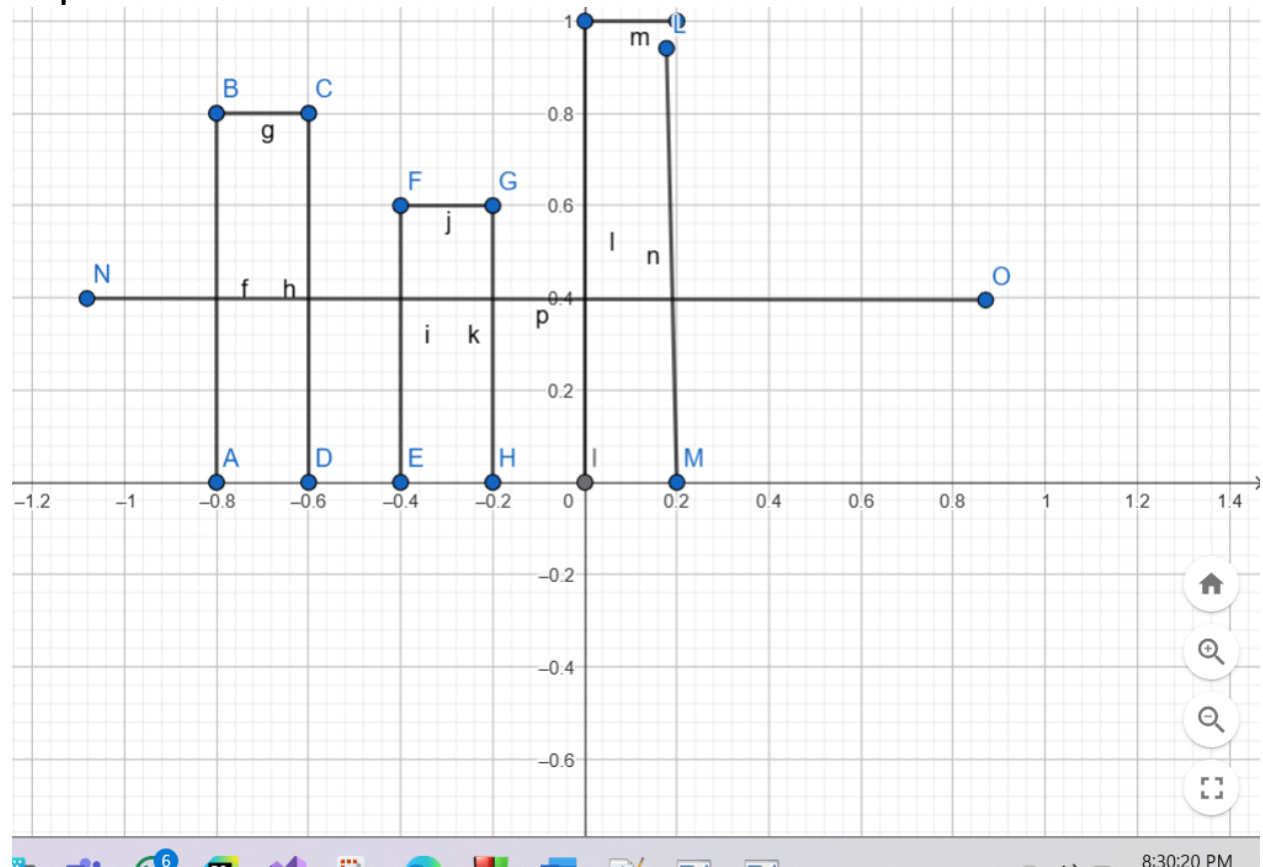
Output Screenshot (Full Screen)-



Question-

Develop a scenario where it will rain and gradually create flood

Graph-



Code- #include <windows.h>

#include <GL/glut.h>

#include <stdlib.h>

#define MAX_DROPS 800

float dropX[MAX_DROPS];

float dropY[MAX_DROPS];

int totalDrops = 0;

float lakeLevel = -0.6f;

float roadFloodLevel = -0.3f;

bool isFlooding = false;

void addDrop() {

if (totalDrops < MAX_DROPS) {

dropX[totalDrops] = -1.0f + static_cast<float>(rand()) / (RAND_MAX / 2.0f);

dropY[totalDrops] = 1.0f;

totalDrops++;

}

```

}

void update(int value) {
    for (int i = 0; i < totalDrops; i++) {
        dropY[i] -= 0.03f;

        if (dropY[i] <= lakeLevel + 0.05f) {
            if (lakeLevel < 0.0f) {
                lakeLevel += 0.0007f; // Fill lake
            } else {
                isFlooding = true; // Lake overflowed
            }

            // Remove the drop
            for (int j = i; j < totalDrops - 1; j++) {
                dropX[j] = dropX[j + 1];
                dropY[j] = dropY[j + 1];
            }
            totalDrops--;
            i--;
        }
    }

    // Add new drops
    for (int i = 0; i < 5; i++) {
        addDrop();
    }

    glutPostRedisplay();
    glutTimerFunc(10, update, 0);
}

void drawRaindrops() {
    glColor3f(0.2f, 0.4f, 1.0f);
    glBegin(GL_LINES);
    for (int i = 0; i < totalDrops; i++) {
        glVertex2f(dropX[i], dropY[i]);
        glVertex2f(dropX[i], dropY[i] - 0.05f);
    }
    glEnd();
}

void drawLakeAndFlood() {
    // Lake (filling)

```

```

glColor3f(0.2f, 0.4f, 0.8f);
glBegin(GL_QUADS);
    glVertex2f(-1.0f, -1.0f);
    glVertex2f(1.0f, -1.0f);
    glVertex2f(1.0f, lakeLevel);
    glVertex2f(-1.0f, lakeLevel);
glEnd();

// Road
glColor3f(0.3f, 0.3f, 0.3f); // asphalt color
glBegin(GL_QUADS);
    glVertex2f(-1.0f, lakeLevel);
    glVertex2f(1.0f, lakeLevel);
    glVertex2f(1.0f, roadFloodLevel);
    glVertex2f(-1.0f, roadFloodLevel);
glEnd();

// Flood water on road
if (isFlooding) {
    glColor4f(0.4f, 0.6f, 0.9f, 0.7f); // semi-transparent blue
    glBegin(GL_QUADS);
        glVertex2f(-1.0f, roadFloodLevel);
        glVertex2f(1.0f, roadFloodLevel);
        glVertex2f(1.0f, lakeLevel + 0.05f);
        glVertex2f(-1.0f, lakeLevel + 0.05f);
    glEnd();
}
}

void drawBuildings() {
    glColor3f(0.7f, 0.7f, 0.7f); // grey buildings
    glBegin(GL_QUADS);
        glVertex2f(-0.8f, -0.3f);
        glVertex2f(-0.6f, -0.3f);
        glVertex2f(-0.6f, 0.4f);
        glVertex2f(-0.8f, 0.4f);

        glVertex2f(-0.4f, -0.3f);
        glVertex2f(-0.2f, -0.3f);
        glVertex2f(-0.2f, 0.3f);
        glVertex2f(-0.4f, 0.3f);

        glVertex2f(0.2f, -0.3f);
        glVertex2f(0.4f, -0.3f);

```

```
        glVertex2f(0.4f, 0.5f);
        glVertex2f(0.2f, 0.5f);
    glEnd();
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    drawLakeAndFlood();
    drawBuildings();
    drawRaindrops();
    glFlush();
}

void init() {
    glClearColor(0.8f, 0.9f, 1.0f, 1.0f); // sky
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-1, 1, -1, 1);
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutCreateWindow("City Lake Flood Simulation");
    init();
    glutDisplayFunc(display);
    glutTimerFunc(0, update, 0);
    glutMainLoop();
    return 0;
}
```

Output Screenshot (Full Screen)-

